

INFORME / DOCUMENTACIÓ

Curs	25/26	Grup	IFC31C	Data	31/05/2026
Mòdul	2526_IFC31C_SAD Seguretat i Alta Disponibilitat				
Alumne/a	Jaume Fuster Valls				
Activitat	Servei web amb certificat protegit amb regles de firewall sobre Docker Swarm per disposar de alta disponibilitat.				



INDEX

PRESENTACIÓ.....	4
ARQUITECTURA DEL SISTEMA.....	5
Descripció de les capes.....	5
Descripció grafica de les capes.....	6
SCRIPTS I FITXERS DE CONFIGURACIÓ.....	7
Script generate_cert.sh.....	7
Dockerfile.toolbox.....	8
Script destroy.sh.....	8
Script deploy.sh.....	9
Script setup_firewall.sh.....	11
Dockerfile de Nginx (dins el directori Nginx).....	13
Configuració nginx.conf (dins el directori Nginx).....	14
COMPROVACIONS I VALIDACIÓ DEL FUNCIONAMENT.....	15
Verificació de l'estat del clúster.....	15
Verificació dels serveis desplegats.....	15
Verificació de la distribució dels serveis.....	16
Comprovació de tolerància a fallades.....	16
Recuperació automàtica dels serveis.....	17
Verificació de la xarxa Overlay.....	18
Comprovació dels ports accessibles.....	19
Comprovació dels ports bloquejats.....	19
Verificació de les regles del tallafocs.....	20
Enllaç al Repositori GitHub.....	20
CONCLUSIONS.....	21

PRESENTACIÓ

Aquest projecte consisteix en el disseny i la implementació d'una infraestructura distribuïda basada en Docker Swarm, utilitzant un entorn virtualitzat de tipus Docker-in-Docker (DinD) per simular múltiples màquines dins d'un únic host físic.

L'objectiu principal és reproduir un entorn de clúster real amb diversos nodes, on els serveis es despleguen de manera automàtica, resiliència i escalable.

L'entorn creat està format per un node manager, encarregat de la gestió i orquestració del clúster, i dos nodes worker, que s'encarreguen de l'execució dels serveis. Sobre aquesta infraestructura s'han desplegat diferents aplicacions en contenidors, concretament un servei web basat en Nginx amb suport HTTPS i un servei de prova SSH per validar la comunicació entre nodes.

Per aconseguir aquest sistema s'ha desenvolupat un conjunt d'scripts d'automatització (`deploy.sh`, `destroy.sh`, `setup_firewall.sh` i `generate_cert.sh`) que permeten crear des de zero tota la infraestructura, incloent la creació del clúster Swarm, la generació de la xarxa overlay, la distribució d'imatges entre nodes i la configuració de certificats per a connexions segures.

A més, s'ha implementat una capa de seguretat mitjançant la configuració de regles de firewall amb iptables, aplicades a cada node del clúster. Aquesta configuració permet controlar el trànsit entrant i sortint, limitant l'accés només als ports necessaris per al funcionament dels serveis desplegats.

L'objectiu principal és entendre i demostrar el funcionament d'un sistema d'orquestració de contenidors en un entorn distribuït, analitzant com Docker Swarm gestiona:

- La distribució de càrrega entre nodes,
- La tolerància a fallades,
- La recuperació automàtica de serveis,
- La consistència del sistema davant de canvis o caigudes de nodes.

També es cerca aplicar conceptes de seguretat en xarxes, mitjançant la implementació de firewall, i validar la comunicació entre serveis utilitzant diferents protocols com HTTP, HTTPS i connexions TCP.

ARQUITECTURA DEL SISTEMA

L'arquitectura implementada es basa en un entorn virtualitzat de tipus Docker-in-Docker (DinD), on una màquina host executa diversos contenidors que simulen nodes independents. Aquests nodes formen un clúster Docker Swarm, encarregat de l'orquestració dels serveis distribuïts.

El sistema està estructurat en diverses capes clarament diferenciades: la capa física (host), la capa de virtualització (Docker DinD), la capa d'orquestració (Swarm), la capa de serveis (contenidors), i la capa de client.

Descripció de les capes

Màquina host

La màquina host és l'entorn físic real on s'executa tot el sistema. Sobre aquesta màquina s'instal·la Docker Engine, que permet crear contenidors Linux completament aïllats. Aquesta capa no forma part del clúster, però és la base de tota la infraestructura.

Capa Docker DinD

Sobre el host es crea una xarxa de contenidors que funcionen com a màquines virtuals:

- node-manager
- node-worker1
- node-worker2

Aquests contenidors executen Docker dins de Docker, permetent simular un entorn distribuït real sense necessitat de múltiples màquines físiques.

Clúster Docker Swarm

El node-manager actua com a Swarm Manager, encarregat de:

- Gestionar l'estat del clúster
- Programar serveis
- Distribuir càrrega entre workers
- Supervisar l'estat dels nodes

Els workers executen els contenidors assignats pel manager.

Xarxa overlay

Els nodes es comuniquen mitjançant una xarxa overlay, que permet:

- Comunicació entre contenidors de diferents nodes
- Resolució per nom de servei (DNS intern Swarm)
- Aïllament respecte la xarxa del host

Això permet que serveis com web_nginx siguin accessibles des de qualsevol node.

Sistema de certificats

S'han generat certificats SSL mitjançant scripts automatitzats i distribuïts a tots els nodes:

- node-manager:/certs
- node-worker1:/certs
- node-worker2:/certs

Aquests certificats permeten habilitar connexions HTTPS segures al servei Nginx.

Capa de serveis (containers)

Sobre el clúster s'executen el serveis web_nginx, servidor web Nginx amb HTTP i HTTPS replicat en múltiples nodes

Capa client

S'utilitza un contenidor client (Alpine Linux) que permet:

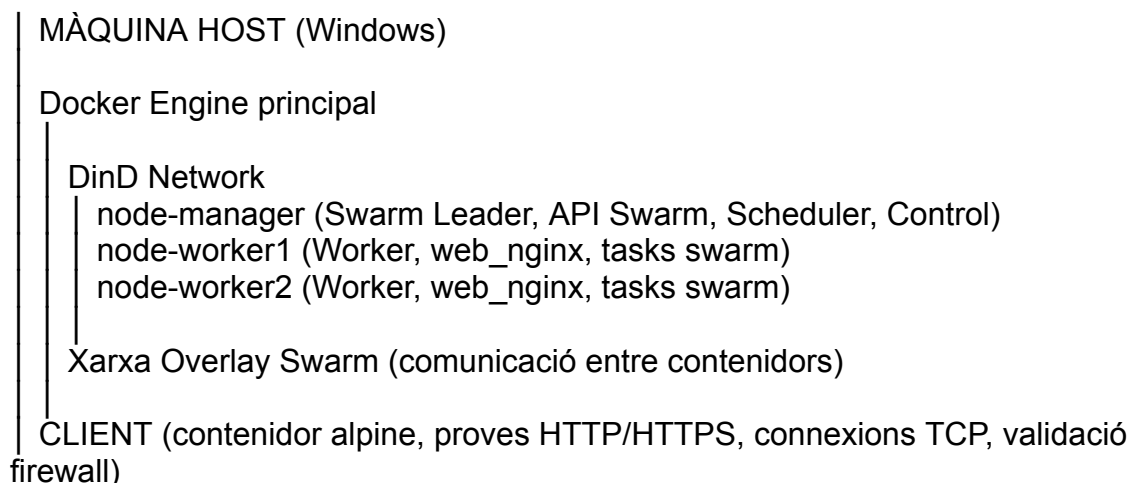
- Fer peticions HTTP i HTTPS
- Provar connectivitat entre serveis
- Validar firewall i ports
- Simular usuaris externs al sistema

Flux de comunicació

1. El client envia una petició HTTP/HTTPS
2. La petició entra a la xarxa overlay
3. Swarm redirigeix la connexió a una rèplica de Nginx
4. El node worker processa la resposta
5. Es retorna la resposta al client

Descripció grafica de les capes

Representació general de l'arquitectura



SCRIPTS I FITXERS DE CONFIGURACIÓ

Per automatitzar la creació de tota la infraestructura s'han desenvolupat diversos scripts i fitxers de configuració. L'objectiu és que el desplegament complet del clúster Docker Swarm es pugui realitzar de manera ràpida, repetible i sense intervenció manual.

Cada fitxer compleix una funció específica dins del projecte. Alguns s'encarreguen de preparar l'entorn, altres de generar certificats de seguretat, crear els nodes del clúster o desplegar els serveis web. També existeixen fitxers dedicats a la configuració del servidor Nginx i a la construcció de les imatges Docker utilitzades durant la pràctica.

A continuació es descriu detalladament la funció de cada component.

Script generate_cert.sh

Aquest script genera els certificats SSL utilitzats pel servidor Nginx.

Codi:

```
#!/bin/bash
set -e
rm -rf certs
mkdir -p certs
openssl req -x509 -nodes -days 365 \
-newkey rsa:2048 \
-keyout certs/servidor.key \
-out certs/servidor.crt \
-subj "/C=ES/ST=Baleares/L=Palma/O=SMX/CN=web_nginx" \
-addext "subjectAltName=DNS:web_nginx,DNS:swarm.local,IP:127.0.0.1"
```

Primer s'eliminen certificats antics i a continuació es genera un certificat autofirmat. Les característiques principals del certificat són:

- Algorisme RSA de 2048 bits.
- Validesa de 365 dies.
- Generació automàtica sense preguntes interactives.
- Inclusió de diversos noms DNS vàlids.

Els fitxers generats són “servidor.crt” (certificat públic) i “servidor.key” (clau privada). Aquests fitxers seran copiats posteriorment a tots els nodes del clúster.

Dockerfile.toolbox

Aquest fitxer defineix una imatge Docker auxiliar que conté les eines necessàries per administrar i interactuar amb l'entorn de pràctiques.

Codi:

```
FROM alpine:3.20
RUN apk add --no-cache \
    bash \
    curl \
    openssl \
    docker-cli \
    git
WORKDIR /work
ENTRYPOINT ["/bin/bash"]
```

La primera instrucció utilitza Alpine Linux com a sistema base. Aquesta distribució destaca per la seva mida reduïda i pel baix consum de recursos.

A continuació s'instal·len les eines necessàries i posteriorment es defineix el directori de treball "WORKDIR /work".

Això permet accedir directament a una consola interactiva quan el contenidor és executat.

Script destroy.sh

Aquest script s'encarrega d'eliminar completament l'entorn de pràctiques.

Codi:

```
#!/bin/bash
docker exec node-manager docker swarm leave --force 2>/dev/null || true
docker rm -f node-manager node-worker1 node-worker2 2>/dev/null || true
docker network rm swarm-net 2>/dev/null || true
echo "Entorno eliminado"
```

La primera instrucció força la sortida del node manager del clúster Swarm, després s'eliminen els tres nodes virtuals creats durant el desplegament i finalment s'elimina la xarxa principal utilitzada per comunicar els nodes.

Aquest script garanteix que qualsevol execució nova del desplegament comenci des d'un entorn completament net.

Script deploy.sh

Aquest és el script principal del projecte. La seva funció és construir tota la infraestructura des de zero.

El procés complet es divideix en diverses fases:

Neteja inicial

```
chmod +x destroy.sh  
./destroy.sh
```

Abans de començar s'elimina qualsevol desplegament anterior.

Creació de la xarxa principal

```
docker network create swarm-net
```

Aquesta xarxa permet que els nodes DinD es comuniquin entre si.

Creació dels nodes

```
docker run -d --privileged --name node-manager ...  
docker run -d --privileged --name node-worker1 ...  
docker run -d --privileged --name node-worker2 ...
```

Es creen tres contenidors Docker-in-Docker (node-manager, node-worker1 i node-worker2)

El mode “--privileged” és necessari perquè cada node pugui executar Docker internament.

Inicialització del clúster Swarm

```
MANAGER_IP=$(docker inspect ...)  
docker swarm init  
docker swarm join --token ...
```

Primer s'obté la IP del manager, posteriorment es crea el clúster i finalment els workers s'uneixen utilitzant un token d'autenticació.
En aquest moment el clúster ja està operatiu.

Creació de la xarxa Overlay

```
docker network create -d overlay xarxa_web
```

Aquesta xarxa virtual permet la comunicació entre serveis desplegats en diferents nodes. És una de les característiques principals de Docker Swarm.

Construcció de la imatge Nginx

```
docker build -t nginx_https ./nginx
```

Es genera una imatge personalitzada que incorpora configuració HTTPS, fitxers HTML i configuració del servidor web.

Distribució de la imatge

```
docker save nginx_https -o nginx.tar  
docker load -i nginx.tar
```

Com que no s'utilitza un registre Docker privat, la imatge es distribueix manualment, això garanteix que tots els workers disposin de la mateixa versió.

Distribució dels certificats

```
docker cp certs/. node-worker1:/certs/
```

Els certificats SSL es copien a tots els nodes i posteriorment seran muntats dins dels contenidors Nginx.

Desplegament del servei web

```
docker service create \  
  --name web_nginx \  
  --replicas 2  
  --constraint 'node.role == worker'
```

Es crea un servei Swarm amb dues rèpliques. Mitjançant la restricció “--constraint” s'impedeix que els contenidors siguin executats sobre el node manager. Això permet reservar el manager exclusivament per a tasques d'orquestració.



Desplegament del client de proves

```
docker service create \  
--name client_test
```

Aquest servei crea un contenidor Alpine que s'utilitza per validar la connectivitat de la xarxa i comprovar el funcionament dels serveis desplegats.

Aplicació del firewall

```
chmod +x setup_firewall.sh  
./setup_firewall.sh
```

Finalment s'apliquen les regles de seguretat als tres nodes del clúster.

Script setup_firewall.sh

L'objectiu d'aquest script és restringir el trànsit entrant únicament als serveis necessaris, reduint així la superfície d'atac del sistema.

La configuració es realitza de manera automàtica sobre els tres nodes del clúster (manager i workers) utilitzant regles d'iptables.

L'script recorre tots els nodes del clúster mitjançant un bucle for. Per a cada node executa una sèrie d'ordres dins del contenidor Docker DinD corresponent, instal·lant i configurant iptables.

Instal·lació d'iptables

```
apk add --no-cache iptables
```

Abans de crear les regles és necessari assegurar-se que el paquet està disponible al sistema Alpine Linux dels nodes, aquest paquet proporciona totes les eines necessàries per gestionar el filtratge de paquets de xarxa.

Eliminació de regles anteriors

```
iptables -F  
iptables -X
```

Abans de crear una nova configuració es netegen totes les regles existents, això garanteix que no existeixin configuracions antigues que puguin interferir amb la nova política de seguretat.

-F elimina totes les regles de les cadenes existents.
-X elimina cadenes personalitzades.



Definició de polítiques per defecte

```
iptables -P INPUT DROP
iptables -P FORWARD ACCEPT
iptables -P OUTPUT ACCEPT
```

A continuació es defineix el comportament general del tallafocs.

Tot el trànsit entrant serà rebutjat excepte aquell que coincideixi amb alguna regla explícita.

(INPUT = DROP) Aquesta és la política més segura perquè funciona sota el principi de "denegar per defecte".

(FORWARD = ACCEPT) Permet el trànsit que Docker necessita per comunicar contenidors, xarxes overlay i serveis Swarm.

OUTPUT = ACCEPT Permet que els nodes puguin iniciar connexions cap a l'exterior, descarregar paquets, comunicar-se amb altres nodes o accedir a Internet.

Permetre el trànsit local

```
iptables -A INPUT -i lo -j ACCEPT
```

La interfície "lo" (loopback) és utilitzada pel propi sistema operatiu per comunicar processos interns, sense aquesta regla moltes aplicacions podrien deixar de funcionar correctament.

Permetre connexions establertes

```
iptables -A INPUT -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
```

Aquesta regla permet la recepció de respostes a connexions iniciades prèviament, sense aquesta regla les respostes serien bloquejades encara que la connexió hagués estat iniciada pel node.

Permetre comunicació entre xarxes internes

```
iptables -A INPUT -s 172.16.0.0/12 -j ACCEPT
iptables -A INPUT -s 10.0.0.0/8 -j ACCEPT
```

Aquestes regles permeten la comunicació interna del clúster, sense aquestes regles els serveis distribuïts deixarien de comunicar-se entre ells.

- La xarxa 172.16.0.0/12 inclou les adreces utilitzades per Docker per connectar els nodes DinD.
- La xarxa 10.0.0.0/8 inclou les adreces de la xarxa overlay creada per Docker Swarm.



Permetre els serveis necessaris

```
iptables -A INPUT -p tcp --dport 22 -j ACCEPT  
iptables -A INPUT -p tcp --dport 80 -j ACCEPT  
iptables -A INPUT -p tcp --dport 443 -j ACCEPT
```

- Permet connexions remotes d'administració (SSH)
- Permet accés web sense xifrar (HTTP)
- Permet l'accés segur al servei Nginx desplegat sobre Docker Swarm (HTTPS)

Registre d'intents bloquejats

```
iptables -A INPUT -m limit --limit 5/min \  
-j LOG --log-prefix "FIREWALL DROP INPUT: "
```

Finalment es registra qualsevol connexió que no compleixi cap regla anterior, la limitació de cinc missatges per minut evita inundar els logs.

Quan es produeix un accés no autoritzat es genera una entrada, aquesta informació és molt útil per a tasques de monitoratge i auditoria.

Dockerfile de Nginx (dins el directori Nginx)

Aquest fitxer construeix la imatge utilitzada pel servei web.

Codi:

```
FROM nginx:latest  
COPY nginx.conf /etc/nginx/nginx.conf  
COPY html/ /usr/share/nginx/html/
```

La imatge base és Nginx oficial. Posteriorment es copia la configuració personalitzada i el contingut web.

Configuració nginx.conf (dins el directori Nginx)

Aquest fitxer defineix el comportament del servidor web.

Codi:

```
events {}
server {
    listen 80;
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl;
    ssl_certificate /etc/nginx/certs/servidor.crt;
    ssl_certificate_key /etc/nginx/certs/servidor.key;
    add_header X-Hostname $hostname;
    location / {
        root /usr/share/nginx/html;
        index index.html;
    }
}
```

Activa la gestió bàsica d'esdeveniments del servidor.

Qualsevol petició HTTP és redirigida automàticament a HTTPS. Això garanteix que tota la comunicació sigui xifrada.

El servei escolta connexions segures al port 443.

Nginx utilitza els certificats generats anteriorment per establir connexions segures. La capçalera permet identificar el node que ha atès la petició. Resulta especialment útil per observar el comportament del clúster i la distribució de càrrega.

Finalment defineix la ubicació dels fitxers web que seran servits als clients. Amb aquesta configuració s'obté un servei web segur, distribuït entre diversos nodes i gestionat automàticament per Docker Swarm.

COMPROVACIONS I VALIDACIÓ DEL FUNCIONAMENT

Una vegada finalitzat el desplegament del clúster Docker Swarm, es van realitzar diverses proves amb l'objectiu de verificar que tots els components funcionaven correctament. Les comprovacions es van centrar principalment en quatre aspectes: l'estat del clúster, el desplegament dels serveis, la tolerància a fallades i el funcionament del tallafocs.

Verificació de l'estat del clúster

La primera comprovació va consistir a verificar que els tres nodes formaven part del mateix clúster Docker Swarm.

Per fer-ho es va executar: `docker node ls`

El resultat mostrava els tres nodes disponibles:

```
/ # docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
sc9lhxnrwhfi7lkmrzaotwfew4 *	node-manager	Ready	Active	Leader	29.5.2
k4hdeuzr5x9bx23v9v9rhmetk	node-worker1	Ready	Active		29.5.2
s0p4fabfy0mhijviieottc2ol	node-worker2	Ready	Active		29.5.2

```
/ #
```

Aquesta informació confirma que:

- El node manager ha estat inicialitzat correctament.
- Els dos workers s'han unit al clúster.
- Tots els nodes es troben en estat Ready.
- El manager és el líder actual del clúster.

Verificació dels serveis desplegats

Posteriorment es van consultar els serveis disponibles.

Per fer-ho es va executar: `docker service ls`

El resultat mostrava els serveis disponibles:

```
/ # docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
y85hbog65174	client_test	replicated	1/1	alpine:latest	
r7xe2701raox	web_nginx	replicated	2/2	nginx_https:latest	*:80->80/tcp, *:443->443/tcp

```
/ #
```

Això indica que:

- El servei web disposa de dues rèpliques actives.
- El contenidor client s'ha desplegat correctament.
- Docker Swarm manté el nombre de rèpliques especificat durant el desplegament.

Verificació de la distribució dels serveis

Per comprovar on s'executaven les rèpliques es va utilitzar:
docker service ps web_nginx

El resultat mostrava les rèpliques següents:

```
/ # docker service ps web_nginx
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR
PORTS						
cox6flf4fvit	web_nginx.1	nginx_https:latest	node-worker2	Running	Running 30 minutes ago	
mius8lx2n80t	web_nginx.2	nginx_https:latest	node-worker1	Running	Running 30 minutes ago	

- web_nginx.1 està al node-worker2
- web_nginx.2 està al node-worker1

Aquesta distribució confirma que el servei web s'executa exclusivament sobre els nodes worker gràcies a la restricció “--constraint” definida durant el desplegament

Comprovació de tolerància a fallades

Una de les característiques principals de Docker Swarm és la seva capacitat per mantenir els serveis operatius davant la caiguda d'un node.

Per verificar-ho es va aturar el primer worker: docker stop node-worker2

Posteriorment es va consultar l'estat del servei: docker service ps web_nginx

El resultat mostrava les rèpliques següents:

```
/ # docker service ps web_nginx
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERR
OR						
wh5fbj0u40h1	web_nginx.1	nginx_https:latest	node-worker1	Running	Running 17 seconds ago	
cox6flf4fvit	_ web_nginx.1	nginx_https:latest	node-worker2	Shutdown	Running 35 minutes ago	
srefo5nw53si	web_nginx.2	nginx_https:latest	node-worker1	Running	Running 17 seconds ago	
pn8b6rjq2l2j	_ web_nginx.2	nginx_https:latest	node-worker2	Shutdown	Running 3 minutes ago	
mius8lx2n80t	_ web_nginx.2	nginx_https:latest	node-worker1	Shutdown	Shutdown 18 seconds ago	

- web_nginx.1 està al node-worker1 amb estat Running
- web_nginx.2 està al node-worker1 amb estat Running
- mentres que el node-worker2 està en estat Shutdown

Docker Swarm va detectar immediatament la pèrdua del node i va recrear automàticament la rèplica desapareguda en l'únic worker disponible.

Aquesta prova demostra la capacitat d'auto recuperació del clúster.

Recuperació automàtica dels serveis

Després de tornar a iniciar el worker2: `docker start node-worker2`

Docker Swarm va recuperar el clúster.

Inicialment les dues rèpliques es van executar sobre un únic worker disponible.

Posteriorment es va forçar una redistribució dels serveis:

`docker service update --force web_nginx`

El resultat mostrava les rèpliques següents:

```
/ # docker service update --force web_nginx
web_nginx
overall progress: 2 out of 2 tasks
1/2: running [=====>]
2/2: running [=====>]
verify: Service web_nginx converged
/ # docker service ps web_nginx
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERR
OR	PORTS					
8je7z4awpg87	web_nginx.1	nginx_https:latest	node-worker2	Running	Running 17 seconds ago	
wh5fbj0u40h1	_ web_nginx.1	nginx_https:latest	node-worker1	Shutdown	Shutdown 17 seconds ago	
cox6flf4fvit	_ web_nginx.1	nginx_https:latest	node-worker2	Shutdown	Shutdown 2 minutes ago	
12t78tu4ht44	web_nginx.2	nginx_https:latest	node-worker1	Running	Running 11 seconds ago	
srefo5nw53si	_ web_nginx.2	nginx_https:latest	node-worker1	Shutdown	Shutdown 12 seconds ago	
pn8b6rjq2l2j	_ web_nginx.2	nginx_https:latest	node-worker2	Shutdown	Shutdown 2 minutes ago	
mius8lx2n80t	_ web_nginx.2	nginx_https:latest	node-worker1	Shutdown	Shutdown 5 minutes ago	

- web_nginx.1 està al node-worker2
- web_nginx.2 està al node-worker1

Aquesta distribució equilibrada demostra el correcte funcionament de l'orquestrador.

Verificació de la xarxa Overlay

Per validar la comunicació interna entre serveis es va utilitzar el contenidor client desplegat dins la xarxa overlay.

```
38d6218519bf:/work# docker exec node-manager docker ps --filter name=client_test
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
d05cc4f2b96f   alpine:latest  "sleep 10000"           About a minute Up About a minute          client_test.1.kfw1tqaer4tkrcu4e96
zgqz0n
38d6218519bf:/work# docker exec -it node-manager sh
/ # docker exec -it d05cc4f2b96f sh
/ # ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=62 time=18.169 ms
64 bytes from 8.8.8.8: seq=1 ttl=62 time=16.165 ms
64 bytes from 8.8.8.8: seq=2 ttl=62 time=17.337 ms
64 bytes from 8.8.8.8: seq=3 ttl=62 time=15.164 ms
^C
--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 15.164/16.708/18.169 ms
```

Des del client es va accedir al servei Nginx utilitzant el seu nom DNS intern amb la comanda següent: `curl http://web_nginx`

Resposta:

```
/ # curl http://web_nginx
<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx/1.31.1</center>
</body>
</html>
```

La resposta confirma que la resolució DNS interna funciona correctament i que Nginx redirigeix automàticament les connexions HTTP cap a HTTPS.

Posteriorment es va provar HTTPS amb: `curl -k https://web_nginx`

Resposta:

```
/ # curl -k https://web_nginx
<!DOCTYPE html>
<html>
<head>
  <title>Docker Swarm HTTPS</title>
</head>
<body>
  <h1>Docker Swarm OK</h1>
  <p>Servidor HTTPS funcionando correctamente</p>
</body>
</html>/ # nc -zv web_nginx 80
web_nginx (10.0.1.2:80) open
/ # nc -zv web_nginx 443
web_nginx (10.0.1.2:443) open
```

Aquesta prova valida simultàniament la xarxa overlay, el DNS intern de Swarm, el servei Nginx, el certificat SSL i la comunicació entre contenidors.

Comprovació dels ports accessibles

Des del contenidor client es van verificar els ports autoritzats.

Port HTTP: nc -zv web_nginx 80

Resposta:

```
/ # nc -zv web_nginx 80
web_nginx (10.0.1.2:80) open
/ #
```

Port HTTPS: nc -zv web_nginx 443

Resposta:

```
/ # nc -zv web_nginx 443
web_nginx (10.0.1.2:443) open
/ #
```

Els dos serveis es trobaven accessibles.

Comprovació dels ports bloquejats

Es van realitzar proves contra ports no autoritzats amb:
nc -zv web_nginx 9999 i no es va obtenir resposta.

També es va provar “curl --connect-timeout 3 http://web_nginx:22” i va tornar: Failed to connect:

```
/ # nc -zv web_nginx 9999
/ # curl --connect-timeout 3 http://web_nginx:22
curl: (7) Failed to connect to web_nginx port 22 after 3 ms: Could not connect to server
/ #
```

Aquest comportament confirma que únicament els ports definits al tallafocs són accessibles.

Verificació de les regles del tallafocs

Per comprovar que les regles s'havien aplicat correctament es va consultar la configuració iptables de cada node.

Al node-manager:

```
38d6218519bf:/work# docker exec -it node-manager iptables -L
Chain INPUT (policy DROP)
target     prot opt source                destination
ACCEPT     all  --  anywhere              anywhere
ACCEPT     all  --  anywhere              anywhere            ctstate RELATED,ESTABLISHED
ACCEPT     all  --  172.16.0.0/12         anywhere
ACCEPT     all  --  10.0.0.0/8            anywhere
ACCEPT     tcp  --  anywhere              anywhere            tcp dpt:ssh
ACCEPT     tcp  --  anywhere              anywhere            tcp dpt:http
ACCEPT     tcp  --  anywhere              anywhere            tcp dpt:https
LOG         all  --  anywhere              anywhere            limit: avg 5/min burst 5 LOG level warn prefix "FIRE"
WALL DROP INPUT: "

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

Chain DOCKER-INGRESS (0 references)
target     prot opt source                destination
RETURN     all  --  anywhere              anywhere
38d6218519bf:/work#
```

Aquesta configuració coincideix exactament amb la política definida durant la implementació.

Enllaç al Repositori GitHub

El projecte complet es troba disponible al següent repositori GitHub, des d'on es poden consultar tots els scripts, configuracions i fitxers utilitzats durant el desenvolupament de la pràctica:

https://github.com/jaumefv981276345/swarm_project.git

Aquest repositori permet reproduir íntegrament la infraestructura descrita en aquest document.

CONCLUSIONS

Durant aquesta pràctica s'ha implementat una infraestructura basada en Docker Swarm utilitzant una arquitectura distribuïda formada per un node manager i dos nodes worker. Tot el procés ha estat automatitzat mitjançant scripts Bash, fet que ha permès desplegar l'entorn complet amb una única execució.

El projecte ha permès aprofundir en conceptes relacionats amb la virtualització lleugera, l'orquestració de contenidors, les xarxes overlay, la distribució de serveis i la tolerància a fallades. A més, s'ha configurat un servidor web Nginx accessible mitjançant HTTPS gràcies a l'ús de certificats SSL autofirmats generats automàticament.

Les diferents proves realitzades han demostrat que Docker Swarm és capaç de redistribuir les rèpliques dels serveis quan un node worker deixa d'estar disponible. Quan es va simular la caiguda d'un worker, les rèpliques es van recrear automàticament en el node restant, garantint la continuïtat del servei. Aquest comportament confirma el correcte funcionament dels mecanismes d'alta disponibilitat proporcionats per Swarm.

Pel que fa a la seguretat, inicialment es va intentar utilitzar Firewalld com a sistema de filtratge de trànsit. Tot i això, els nodes DinD utilitzats en la pràctica no disposaven dels serveis necessaris per al seu funcionament, especialment el servei DBus requerit per Firewalld. Aquesta limitació provocava errors durant la configuració i feia inviable la seva utilització dins dels contenidors.

Per aquest motiu es va optar per utilitzar iptables, una solució més simple però totalment funcional dins d'aquest entorn. Mitjançant iptables es van definir polítiques restrictives d'accés, permetent únicament les connexions necessàries per al funcionament dels serveis web i la gestió dels nodes. També es van incorporar regles de registre per facilitar la detecció d'intents d'accés no autoritzats.

Malgrat que la infraestructura funciona correctament, existeixen diversos punts de fallada potencials. El principal és la presència d'un únic node manager. Si aquest node deixés de funcionar, el clúster continuaria executant els serveis existents, però no seria possible administrar-lo ni desplegar nous serveis fins a recuperar-lo. En entorns de producció seria recomanable disposar de diversos managers per evitar aquest problema.

Un altre punt de millora és l'ús de certificats autofirmats. Encara que són adequats per a un entorn de laboratori o proves, en un sistema real s'haurien d'utilitzar certificats emesos per una autoritat de certificació reconeguda per garantir la confiança dels clients.

Finalment, aquesta pràctica ha permès integrar en un únic projecte coneixements de xarxes, seguretat, contenidors i administració de sistemes, demostrant com és possible desplegar una infraestructura distribuïda funcional, segura i capaç de recuperar-se davant determinades situacions de fallada.